

TechWing Daily Log — Database Upgrade Prompt

TECHWING DAILY LOG — DATABASE UPGRADE
Google Antigravity Implementation Prompt

I have an existing Python Flask web application called TechWing Daily Log / Login App.

I want you to upgrade its current JSON-based database into a proper production-ready SQL database, while preserving the existing

1. FIRST: INSPECT THE EXISTING PROJECT

Before changing anything, inspect the complete project structure and understand how the current application works.

The existing project contains files similar to:

- app.py
- database.json
- index.html
- templates/index.html
- static/app.js
- static/styles.css
- static/manifest.json
- static/sw.js
- requirements.txt

The current backend is Python Flask.

DO NOT replace Flask unless there is a genuine technical requirement.
DO NOT redesign the frontend.
DO NOT remove existing features.
DO NOT change existing field names unnecessarily.

2. CURRENT DATABASE STRUCTURE

The existing database.json contains:

- users
- logs
- suggestions

USERS — CURRENT FIELDS

id
email
name
combo
password

LOGS — CURRENT FIELDS

id
user_email
combo
date
day
lab
checkIn
checkOut
trainer
topics
practical
assignment
doubts

SUGGESTIONS — CURRENT FIELDS

id
date
name
email
category
subject
message
target_whatsapp
status

Preserve the meaning of all existing fields.

3. CREATE A PROPER SQL DATABASE

Replace database.json as the PRIMARY persistence mechanism.

Use:

- SQLAlchemy ORM + Flask
- MySQL for development
- PostgreSQL compatibility for production

Use an environment variable for the database URL, for example:

DATABASE_URL=mysql+pymysql://username:password@localhost/techwing_daily_log

Do NOT hardcode database credentials.

4. USERS TABLE

Create:

users

Fields:

id
name
email
combo
password_hash
role
is_active
created_at
updated_at
last_login_at

Recommended:

id: BIGINT/INTEGER PRIMARY KEY
name: VARCHAR
email: VARCHAR UNIQUE NOT NULL
combo: VARCHAR NOT NULL
password_hash: VARCHAR NOT NULL
role: VARCHAR NOT NULL DEFAULT 'USER'
is_active: BOOLEAN NOT NULL DEFAULT TRUE
created_at: DATETIME/TIMESTAMP
updated_at: DATETIME/TIMESTAMP
last_login_at: DATETIME/TIMESTAMP NULL

Roles:

USER
ADMIN

Migrate the existing owner/admin account appropriately.

5. PASSWORD SECURITY

The current application stores passwords in database.json.

Do NOT continue storing plaintext passwords.

Use Argon2, bcrypt, or Flask/Werkzeug secure password hashing.

NEVER store plaintext passwords in SQL.

Do not simply copy existing plaintext passwords into password_hash. If existing passwords cannot be safely converted, require a

6. DAILY LOG TABLE

Create:

daily_logs

Fields:

id
user_id
user_email
combo
log_date
day

lab
check_in
check_out
trainer
topics
practical
assignment
doubts
created_at
updated_at

Use:
user_id -> users.id

Keep user_email if the existing frontend/API depends on it, but use user_id as the relational connection.

The existing API should continue returning:

id
user_email
combo
date
day
lab
checkIn
checkOut
trainer
topics
practical
assignment
doubts

Map SQL names such as log_date, check_in, check_out back to the existing API names.

7. LOG ID COMPATIBILITY

Preserve compatibility with existing log IDs.

Do not destroy existing IDs during migration.

8. SUGGESTIONS TABLE

Create:
suggestions

Fields:
id
date
name
email
category
subject
message
target_whatsapp
status
created_at
updated_at

Add useful indexes on:
email
date
status
category

9. LOGIN ACTIVITY TABLE

Create:
login_activity

Fields:
id
user_id
login_time
logout_time
created_at

Optional:
ip_address
user_agent

Only collect these if genuinely required for security/administration. Do not implement hidden device surveillance.

10. AUDIT LOG TABLE

Create:
audit_logs

Fields:
id
user_id
action
target_type
target_id
metadata
created_at

Possible actions:
LOGIN
LOG_CREATED
LOG_UPDATED
SUGGESTION_CREATED
PROFILE_UPDATED
ADMIN_VIEW
USER_ACTIVATED
USER_DEACTIVATED

11. DATABASE RELATIONSHIPS

Use:
users
|
|-- daily_logs
|-- login_activity
|-- audit_logs

Suggestions may reference user_id where appropriate while preserving compatibility.

12. INDEXES

users:
UNIQUE(email)
INDEX(combo)
INDEX(role)
INDEX(is_active)

daily_logs:
INDEX(user_id)
INDEX(user_email)
INDEX(combo)
INDEX(log_date)
INDEX(user_id, log_date)

suggestions:
INDEX(email)
INDEX(date)
INDEX(category)
INDEX(status)

login_activity:
INDEX(user_id)
INDEX(login_time)

audit_logs:
INDEX(user_id)
INDEX(action)
INDEX(created_at)

13. EXISTING API COMPATIBILITY

Do NOT unnecessarily change these routes:

```
GET /api/config
POST /api/register
POST /api/login
GET /api/logs
POST /api/logs
POST /api/ai/parse-voice
GET /api/suggestions
POST /api/suggestions
GET /api/owner/master
```

The SQL database should replace JSON persistence underneath these APIs.

14. REGISTRATION

Keep:
POST /api/register

Request should continue supporting:

```
{
  "email": "...",
  "name": "...",
  "combo": "...",
  "password": "..."
}
```

Internally:
password -> secure hash -> password_hash -> users table

Never return password or password_hash.

15. LOGIN

Keep:
POST /api/login

Verify email/password against the secure password hash.

On successful login:
- update last_login_at
- create login_activity record
- optionally create LOGIN audit record

Never return password or password_hash.

16. LOG APIs

Keep:
GET /api/logs
POST /api/logs

Existing filters must continue to work:
email
combo

Examples:
GET /api/logs?email=student@example.com
GET /api/logs?combo=genai-aws

Do not break these filters.

17. DAILY LOG SAVE/UPDATE

Preserve creation and updating behavior.

Existing fields:
id
user_email
combo
date
day
lab
checkIn
checkOut

trainer
topics
practical
assignment
doubts

Do not create duplicate records when an update was intended.

18. VOICE AI API

Preserve:
POST /api/ai/parse-voice

Do not remove or break the existing voice/AI parsing functionality.

19. SUGGESTIONS API

Preserve:
GET /api/suggestions
POST /api/suggestions

Continue supporting:
name
email
category
subject
message
target_whatsapp
status
date

20. OWNER / ADMIN MONITORING

The current application has:
GET /api/owner/master

Replace weak/hardcoded passcode authorization with proper authentication and role-based authorization.

Do NOT continue relying on hardcoded admin secrets.

Use ADMIN role + secure authentication.

21. ADMIN MONITORING

Admin should be able to retrieve authorized application data such as:

Users:
- Name
- Email
- Combo
- Active/Inactive
- Created date
- Last login
- Number of logs

Daily logs:
- Authorized daily log records

Login activity:
- User
- Login time
- Logout time

Suggestions:
- Name
- Email
- Category
- Subject
- Message
- Date
- Status

Audit activity:

- Important application actions

22. USER DATA ISOLATION

Normal users must not access another user's private records by changing URL parameters.

Backend authorization must verify the authenticated user's identity and permissions.

USER:

- own profile
- own logs
- own activity

ADMIN:

- authorized administrative access to application records

Authorization must be enforced by the backend, not only JavaScript.

23. DATABASE MIGRATION

Create a migration script that reads database.json and imports:

- users
- logs
- suggestions

Before migration, back up database.json.

Do not lose existing data.

Make migration repeatable/idempotent where practical and prevent duplicates.

24. EXISTING DATA PRESERVATION

Existing users, logs, and suggestions must remain available after migration.

Do not discard:

user_email
combo
date
day
lab
checkIn
checkOut
trainer
topics
practical
assignment
doubts

25. DATABASE MIGRATIONS

Use Flask-Migrate or Alembic.

Support commands equivalent to:

flask db init
flask db migrate
flask db upgrade

26. ENVIRONMENT VARIABLES

Create:

.env.example

Include placeholders such as:

DATABASE_URL=
SECRET_KEY=
ADMIN_EMAIL=

Do NOT commit real secrets.

Add .env to .gitignore.

27. REQUIREMENTS

Update requirements.txt with only required packages, such as:

Flask
SQLAlchemy
Flask-SQLAlchemy
Flask-Migrate
PyMySQL
python-dotenv
Werkzeug

28. SECURITY

Implement:

- secure password hashing
- role-based authorization
- environment-based secrets
- backend authorization
- SQLAlchemy/parameterized queries
- input validation
- safe error handling
- appropriate CORS configuration

Never expose passwords, password hashes, database credentials, or secret keys.

29. DO NOT BREAK THE FRONTEND

Do not redesign:

index.html
templates/index.html
static/app.js
static/styles.css

unless a small compatibility change is necessary.

Goal:

SAME UI
SAME USER EXPERIENCE
SAME API ROUTES
STRONGER DATABASE

30. JSON MUST NO LONGER BE THE LIVE DATABASE

After successful migration, database.json must not be used as the live database.

Architecture:

Frontend



Flask REST API



SQLAlchemy



MySQL / PostgreSQL

database.json should remain only as migration source/backup if needed.

31. DATABASE CONNECTION

Use SQLALCHEMY_DATABASE_URI from environment variables.

Do not hardcode database credentials.

32. TRANSACTIONS

Use proper transactions and rollback handling.

Registration:

INSERT -> COMMIT

Update:

UPDATE -> COMMIT

Failure:

ROLLBACK

33. VALIDATION

Validate:

Users: email, name, combo, password

Logs: user, date, lab, checkIn, checkOut

Suggestions: subject, message

Return proper HTTP status codes.

34. API RESPONSE COMPATIBILITY

Keep frontend-friendly response names:

id

user_email

combo

date

day

lab

checkIn

checkOut

trainer

topics

practical

assignment

doubts

Map internal SQL names to these API names.

35. TEST EVERYTHING

Test:

POST /api/register

POST /api/login

POST /api/logs

GET /api/logs

GET /api/logs?email=...

GET /api/logs?combo=...

GET /api/suggestions

POST /api/suggestions

POST /api/ai/parse-voice

GET /api/owner/master

Also test unauthorized access.

36. DATA INTEGRITY TESTS

Verify:

- duplicate email cannot create duplicate user
- user cannot read another user's logs
- admin can access authorized administrative data
- disabled users cannot authenticate
- logs reference valid users
- migrated data is intact

37. BACKUP

Before migration:

database.json -> backup -> SQL migration

Do not delete the original JSON until migration is verified.

38. FUTURE ANDROID / IOS COMPATIBILITY

Design the backend as a REST API for:

Android

iOS

Web

Mobile applications must communicate with the deployed Flask backend over HTTPS.

Do NOT allow mobile apps to connect directly to the database.

Architecture:
Android / iOS / Web
↓
HTTPS
↓
Flask REST API
↓
SQLAlchemy
↓
MySQL/PostgreSQL

39. NO HIDDEN SURVEILLANCE

Admin monitoring may include legitimate application activity:

- login activity
- daily logs
- suggestions
- account status
- important application actions

Do NOT monitor:

- phone files
- camera
- microphone
- contacts
- SMS
- location
- other applications

unless explicitly required and appropriately disclosed/authorized.

40. FINAL REPORT

After implementation, report:

A. Database architecture

B. Tables created:

users
daily_logs
suggestions
login_activity
audit_logs

C. Migration results:

- users migrated
- logs migrated
- suggestions migrated

D. Every file modified/created

E. Dependencies added

F. Environment variables required

G. Exact Windows database/setup/migration/run commands

H. API test results

I. Security improvements

J. Remaining work

Clearly separate:

DATABASE COMPLETE
CLOUD DEPLOYMENT REQUIRED
ANDROID CONVERSION REQUIRED
IOS CONVERSION REQUIRED

Do not claim anything is complete unless it was actually implemented and tested.

MOST IMPORTANT RULES

1. Do not delete existing user data.
2. Do not delete existing daily logs.
3. Do not delete existing suggestions.
4. Do not redesign the UI.
5. Do not break existing API routes.
6. Do not store plaintext passwords.
7. Do not hardcode admin passcodes.
8. Do not use database.json as the live database after migration.
9. Do not expose password hashes through APIs.
10. Do not allow users to access other users' logs by changing URL parameters.
11. Do not introduce hidden device surveillance.
12. Keep the application Android/iOS-ready.
13. Back up database.json before migration.
14. Test everything after migration.
15. If an existing feature conflicts with this migration, explain the conflict before removing or changing it.

IMPLEMENT THE DATABASE UPGRADE FIRST.

DO NOT START ANDROID CONVERSION YET.

Immediate goal:

Upgrade the existing TechWing Daily Log application's JSON database to a secure relational SQL database while preserving the cu